KESSLER

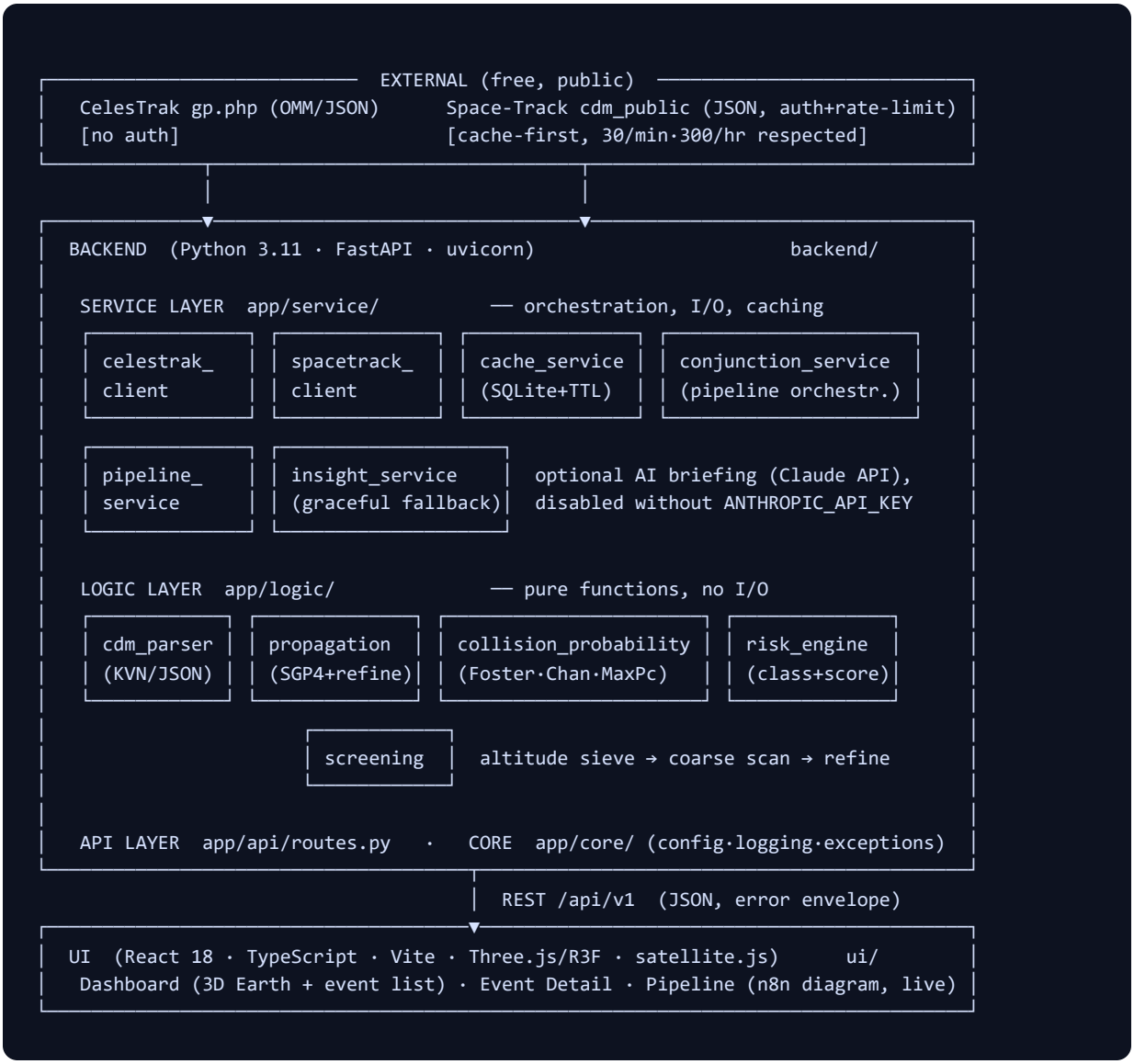
High-Level Design (HLD)

Document ID: KSL-HLD-001 **Version:** 1.0 **Date:** 21 July 2026
Traces to: KSL-SRS-001

1. System Overview

KESSLER is a two-tier web system — a Python computation/orchestration backend and a React 3D frontend — arranged around a **cache-first data pipeline**. External space-surveillance sources are polled on schedule (never per user request), normalised into a local store, enriched by the risk engine, and served to clients. The pipeline is modelled as six named **agents**; the same agent graph is rendered as the n8n-style diagram shipped in `documents/diagrams/` and embedded live in the UI's Pipeline page.

2. Architecture



2.1 Component Responsibilities

Component	Layer	Responsibility	Key SRS trace
`celestrak_client`	service	Fetch OMM JSON groups (active, stations, custom NORAD sets); retry with backoff; typed `DataSourceError` on failure.	FR-ING-1
`spacetrack_client`	service	Authenticated session; enforced min-interval throttle + hourly budget; `cdm_public` and `gp` queries; hard-disabled without credentials.	FR-ING-2, CON-1
`cache_service`	service	SQLite-backed key/TTL store for GP sets and CDM batches; last-known-good fallback; swap-out point for AWS.	FR-ING-3/5, NFR-5
`conjunction_service`	service	Pipeline orchestrator: load (live|demo) → parse → Pc → risk → persist snapshot; exposes read model to API.	FR-CDM-*, FR-RSK-*
`pipeline_service`	service	Tracks per-agent stage status/timings/counters powering the live diagram.	FR-UI-3
`insight_service`	service	Optional Claude-generated analyst briefing; deterministic template fallback.	FR-RSK-4
`cdm_parser`	logic	CCSDS KVN + cdm_public JSON → typed `Cdm`; strict validation.	FR-CDM-1..4
`propagation`	logic	SGP4 wrapper (TEME), state sampling, TCA refinement via bounded minimisation.	FR-SCR-3
`collision_probability`	logic	Encounter-plane projection; Foster 2D numerical Pc; Chan analytic series; conservative max-Pc.	FR-PC-1..5
`screening`	logic	Catalogue screening: altitude-band sieve → coarse sampling → refinement.	FR-SCR-1..5
`risk_engine`	logic	Risk classification, urgency scoring, recommended actions.	FR-RSK-1..3

2.2 The Agent Pipeline

The runtime data flow is modelled as six agents; each maps to concrete modules above. This is the graph shown in the n8n-style diagram (KSL-DIA-001):

#	Agent	Trigger	Function
1	**Catalog Sync**	Startup + hourly	Pull OMM catalogue (CelesTrak / demo file) → cache.
2	**CDM Ingest**	Startup + 30 min	Pull cdm_public batch (Space-Track / demo file) → cache.
3	**Parser & Validator**	On new batch	Normalise raw rows/KVN into typed CDMs; quarantine invalid records.
4	**Pc Engine**	Per event	Foster + Chan Pc (or max-Pc), method-divergence check.
5	**Risk Analyst**	Per event	Classification, urgency, action text, optional AI briefing.
6	**Publisher**	On snapshot	Persist read-model, expose via REST, notify UI.

3. Data Design (summary)

SQLite (file `backend/app/data/cache.db`) with three tables: `kv_cache` (source payload cache with TTL), `events` (enriched conjunction read-model, JSON document per event), `pipeline_runs` (agent, started/finished, status, items, error). Full DDL in LLD §4. Rationale: zero-dependency local store; the service interface (`CacheService`) is the single seam for the later AWS move (DynamoDB/RDS + S3).

4. Error Handling & Logging Strategy

- **Exception hierarchy** rooted at `KesslerError(code, message, http_status)`: `DataSourceError` (502), `RateLimitError` (503), `CdmParseError` (422), `PropagationError` (500), `NotFoundError` (404), `ConfigError` (500).
- **Boundary rule**: logic layer raises; service layer catches, contextualises, decides fallback (cache, demo, skip-record) and re-raises typed errors only when unrecoverable; API layer converts to the JSON error envelope via global handlers. UI never receives a stack trace.
- **Logging**: `logging` with console + rotating file handlers; per-request request_id middleware; agent runs logged start/finish with duration and item counts. WARN = degraded (fallback used), ERROR = user-visible failure.

5. Technology Choices

Choice	Rationale
FastAPI + Pydantic v2	Typed request/response contracts, free OpenAPI docs, async I/O for polite external fetching.
python-sgp4	The community-standard, battle-tested propagator (same core as CelesTrak's).
NumPy (+ pure-py quadrature)	Vectorised screening and Pc integration without heavyweight dependencies.
SQLite	Zero-ops local persistence; interface-isolated for AWS later.
React + react-three-fiber + drei	Declarative Three.js; the 3D scene is a component tree, testable and maintainable.
satellite.js (UI)	Client-side SGP4 for smooth 60 fps orbit animation without hammering the API.
Vite	Fast dev server with `/api` proxy to backend (no CORS pain in dev).

6. Deployment View

v1.0 (now): localhost — `uvicorn app.main:app --port 8000` and `npm run dev` (Vite, :5173, proxying `/api`). No Vercel deployment. **v2.0 (later, when AWS is provided):** backend container on ECS/Lightsail; EventBridge schedules replace in-process scheduler; S3 or RDS replaces SQLite behind `CacheService`; CloudWatch replaces file logs. No logic-layer change anticipated.

7. Risks & Mitigations

Risk	Impact	Mitigation
Space-Track account suspension (rate misuse)	Loss of live CDMs	Central throttle + budget guard; cache-first; demo fallback.
cdm_public lacks covariance	No full-fidelity Pc	Max-Pc path (FR-PC-3) with explicit `pc_type` labelling; KVN parser ready for operator CDMs.
SGP4 accuracy limits (km-level at days)	Screening false ±	Present screening as triage, not operational CA; document error budget in UI.
WebGL performance on low-end devices	Choppy demo	Cap orbit segments, requestAnimationFrame-driven, degrade to 2D list gracefully.